From SBOM to CRA Realities

Turning Component Inventories into **'Critical + Actively Exploited'**

Andrey Lukashenkov | Vulners

NSSS Spring 2026 | Friday SBOM Focus | April 10, 2026

Disclaimer

I am not a lawyer. I am definitely not ***your*** lawyer.

This talk is an engineer's reading of publicly available regulatory documents. It is not legal advice. Consult qualified Consult qualified counsel for compliance decisions.

The Obligation

Sept 11, 2026

Art. 14 reporting goes live.
You must report actively exploited vulnerabilities in your products.

Continuous monitoring

Art. 13(7): systematic documentation of vulnerabilities using third-party intelligence.

24h to report

Once you become aware of active exploitation. This is the reporting window, not the discovery deadline.

The real question isn't how fast you can report.

It's whether you have a defensible process that allows you to **become aware at all**. If a vulnerability in your product is actively exploited and you had no process to detect that — "we didn't know" is not a defense.

You have this identifier in your SBOM.

```
pkg:deb/ubuntu/openssl@1.1.1f-1ubuntu2.24?arch=amd64&distro=ubuntu-20.04
```

In five months, you need to continuously answer:

1. Is this vulnerable? Art. 3(40)
2. Is it exploitable in your product? Art. 3(41)
3. Is it being actively exploited right now? Art. 3(42)

Not once. For every component. Continuously. This talk is about what building that process actually looks like.

What the CRA Actually Demands

Three tiers of vulnerability — three different obligations

Art. 3(40)	**Vulnerability** Weakness that can be exploited	**Track & document**
Art. 3(41)	**Exploitable Vulnerability** Realistic exploitation under practical operational conditions	**Cannot ship with known exploitable vulns**
Art. 3(42)	**Actively Exploited** Reliable evidence of malicious exploitation in a system	**24h early warning → 72h notification → 14-day report**

Two Worlds, One Seam

World 1: SBOM Ecosystem

"What's in my software?"

Identifiers, formats, tools
PURL, CPE, CycloneDX, SPDX
Syft, Trivy, cdxgen

World 2: Vuln Intelligence

"What's dangerous right now?"

CVE List, NVD, EUVD
Vendor advisories, exploit feeds
KEV, EPSS, threat intel

THE SEAM: component-to-vulnerability matching. The CRA requires you to connect these two worlds continuously — and this is where all the friction lives. That's where this talk goes deep.

What the Developer Wrote vs What Ships

Same application, same scanner (Syft), same format (CycloneDX 1.6)

github.com/latiotech/insecure-kubernetes-deployments/tree/main/insecure-app

Repository SBOM

What the developer wrote

5 packages

flask
cryptography
requests
lxml
werkzeug

Image SBOM

What actually ships

677 packages

294 deb packages
(Ubuntu 20.04 base image)

352 Go modules
(from docker-ce-cli — one apt-get line)

31 Python packages
(5 yours + 26 transitive)

Under the CRA, if it ships in the product, it's part of the vulnerability surface.

What's Actually Inside the Image SBOM?

insecure-app | Ubuntu 20.04 | Syft 1.42.1 | CycloneDX 1.6

11,725
SBOM entries

677
with PURL

3
ecosystems

0%
Go dep edges

Ecosystem	Packages	Dep edges	CPE quality
deb (Ubuntu)	294	Rich	Good (distro suffix issue)
Python (pip)	31	Partial (9/31)	Hit-or-miss
Go modules	352 (210 unique)	Zero	Broken (escaped slashes)

ENISA SBOM Guide, Table 13: 80% minimum / 95% target for transitive dependency coverage. Go: 0%.

The Identifier Zoo

One SBOM, three ecosystems, many identifier formats

deb (Ubuntu)

PURL pkg:deb/ubuntu/openssl@1.1.1f-1ubuntu2.24?distro=ubuntu-20.04

CPE cpe:2.3:a:openssl:openssl:1.1.1f-1ubuntu2.24:*:*:*:*:*:*:*

Same package, two naming systems, different version strings

Python

PURL pkg:pypi/cryptography@3.3.2

CPE cpe:2.3:a:cryptography.io:cryptography:3.3.2:*:*:*:*:python:*:*

PURL is clean. CPE vendor varies by who registered it

Go modules

PURL pkg:golang/golang.org/x/net@v0.0.0-20210226172049-e18ecbb05110

CPE cpe:2.3:a:golang:x\/net:v0.0.0-20210226172049...:*:*:*:*:*:*:*

Pseudo-version timestamp, escaped slashes, no stable mapping

Which identifier works with which vulnerability database? That's Part 2.

The Seam

openssl — The Ecosystem Mismatch

openssl@1.1.1f-1ubuntu2.24 | 8 CVEs from OSV API | Grype says: clean

Your SBOM PURL

pkg:deb/ubuntu/openssl@1.1.1f-1ubuntu2.24
?distro=**ubuntu-20.04**

Maps to ecosystem: Ubuntu:20.04:LTS
Standard support ended April 2025

OSV advisory PURL

pkg:deb/ubuntu/openssl@1.1.1f-1ubuntu2.24
?distro=**esm-infra/focal**

Maps to ecosystem: Ubuntu:Pro:20.04:LTS
Fix: 1.1.1f-1ubuntu2.24+esm2 (requires Pro)

What happens:

OSV has no Ubuntu:20.04:LTS entry for these advisories — only Ubuntu:Pro:20.04:LTS.
Grype looks for Ubuntu:20.04:LTS entries → finds none → reports clean.
Grype itself warns: "EOL distro — vulnerability data may be incomplete."
The vulnerabilities are real. The advisory just doesn't exist in the non-Pro ecosystem.

The seam: same package, same version, different distro qualifier → different vulnerability visibility.

The Seam

openssl — Two Types of Findings

Same OSV query returns both. Your pipeline needs to process them differently.

Type 1: Direct CVE Match

High confidence — actionable as-is

UBUNTU-CVE-2025-9230

OSV matched this CVE directly against your PURL. Version-aware. The database says: this specific vulnerability affects your specific package version.

CMS decrypt → DoS / RCE
Feed to EPSS, check KEV, triage.

Type 2: Advisory Bundle

Needs unpacking — not all CVEs apply

USN-7786-1 (3 CVEs in **related**)

CVE-2025-9230 also has direct match ✓

CVE-2025-9231 no direct match — *"25.04 only"*

CVE-2025-9232 no direct match — *"25.04 only"*

OSV got this right: no individual match for 9231/9232. But the USN still lists them.

The call: Do you triage advisory-bundled CVEs the same as direct matches? If you don't unpack them, you over-count. If you ignore them, you might miss one that applies.

Commission §213: "must investigate whether a reported vulnerability is exploitable in practice." Your pipeline needs logic for both finding types.

Database Monitoring ≠ Scanning

What the regulators actually require

Commission Guidance §211

*"relevant publicly accessible vulnerability databases, such as the **European vulnerability database [EUVD]** ... or other prominent vulnerability databases, such as the **Common Vulnerability and Exposures (CVE) List**."*

ENISA Package Managers advisory §4.3.2

*"Subscribe to and monitor public vulnerability feeds: **EUVD, OSV.dev, Snyk, or NVD**"*

Recommended as a separate control from "Automate vulnerability scanning in CI/CD"

The openssl lesson: OSV.dev (database, named in ENISA examples) found 8 CVEs.
Grype (scanner) found 0.
A scanner alone might not be a defensible process under the CRA.

cryptography — Multi-Source Divergence

cryptography@3.3.2 | Python | Same package, four databases, four answers

PURL → OSV.dev

10 advisories (version-aware)

CVE-2023-23931 ✓
CVE-2023-49083 ✓
CVE-2023-50782 ✓
CVE-2026-26007 ✓
CVE-2023-0286 OSV only
CVE-2024-0727 OSV only
CVE-2026-34073 OSV only

+ 3 GHSA-only advisories

CPE → NVD

4 CVEs (version-aware)

CVE-2023-23931 ✓
CVE-2023-49083 ✓
CVE-2023-50782 ✓
CVE-2026-26007 ✓

Missing:
CVE-2023-0286 (OpenSSL)
CVE-2024-0727 (PKCS12)
CVE-2026-34073 (new)

No GHSA — NVD doesn't index them

CPE → vulnerability-lookup

9 CVEs (no version filter!)

4 correct matches ✓

5 false positives:
CVE-2016-9243 (< 1.5.3)
CVE-2020-25659 (<3.2)
CVE-2020-36242 (< 3.3.2)
CVE-2023-38325 (≥0.8)
CVE-2024-26130 (≥38.0)

Vendor:product match, no version logic

Grype agrees with OSV (10 advisories) — it uses GHSA for Python, not CPE. EUVD has no CPE/PURL search; CIRCL is the closest proxy.

> **Commission §211 names EUVD as monitoring source.** But EUVD has no API search by CPE or PURL. vulnerability-lookup (CIRCL/GCVE) adds CPE search — without version matching.

Go Modules — Scale + Reachability

352 Go entries | 210 unique packages | 141 duplicated across binaries | 0 dep edges

178 properly versioned → matchable via PURL | **32 pseudo**

Grype Finds (178 matchable)

37 unique Go vulns (2 critical, 15 high)

stdlib: 26 CVEs grpc: 1 critical

containerd, x/crypto, oauth2, otel…

Version conflicts in the same image:

grpc v1.69 + v1.71, otel v1.31 + v1.34
Which binary uses which? SBOM can't say.

The Reachability Question

The app is Python + Flask.
docker-ce-cli is build tooling.

Are 37 Go vulnerabilities reachable at runtime? Almost certainly not.

But without dep edges or reachability analysis, you can't prove it — and the CRA asks you to.

Commission §213: "The mere fact that a vulnerability is reported as exploitable does not, in itself, mean that it is exploitable in practice." Reachability analysis is the Art. 3(41) filter.

Even the 178 "properly versioned" packages have a CPE problem →

golang.org/x/net — Lost in Translation

Properly versioned (v0.39.0), duplicated across binaries, invisible to CPE-based tooling

PURL pkg:golang/golang.org/x/net@v0.39.0

CPE cpe:2.3:a:golang:networking:v0.39.0:*:*:*:*:go:*:*

2	0	2	0
OSV advisories (PURL query)	NVD matches (CPE query)	CIRCL false positives (CPE, no version filter)	Grype findings (scanner miss)

golang.org/x/net → golang:networking — the SBOM tool loses the namespace. NVD has no idea what that is.

CVE-2025-47911 + CVE-2025-58190:
real vulns, only OSV finds them via PURL.

CIRCL finds 2 CVEs (2023-3978, 2023-44487) —
both false positives, wrong versions, no overlap with OSV.

Grype misses both real vulns
— despite x/net being properly versioned (v0.39.0), not pseudo-versioned.

CRA Taxonomy → Tooling Reality

What each CRA definition requires — and where the tooling breaks

CRA Definition	OSS Path (PURL)	Proprietary / CPE Path	Gap
Art. 3(40) Known vulnerability	**PURL → OSV:** CVSS, fix refs, GHSA — no CVE hop needed ✓ direct from PURL query	**CPE → NVD:** CVSS via CPE match → CVE ID Vendor advisories for closed-source	Go CPE broken EUVD: no PURL/ CPE search
Art. 3(41) Exploitable vulnerability §213 filter	**EPSS** (CVE-indexed, need alias) **Reachability:** Eclipse Steady (Java), rest proprietary **VEX:** OpenVEX any ID, product = PURL ✓	**EPSS** (CVE-indexed) No source → no call graph **VEX:** CSAF VEX typically CVE, vendor-defined products	EPSS needs CVE Exploit PoCs: feed into EPSS, not a standalone metric VEX maturing
Art. 3(42) Actively exploited	**CISA KEV:** CVE-indexed Must extract CVE alias from OSV/GHSA record CERT advisories, own telemetry, threat intel	**CISA KEV:** CVE-indexed Natural fit for CPE → NVD → CVE path Vendor PSIRT, CERT feeds	KEV needs CVE Evidence = manual assessment

Art. 3(40): PURL works without CVE for OSS. Art. 3(41): EPSS + reachability + VEX — EPSS needs CVE, reachability has no open standard. Art. 3(42): KEV + evidence, all CVE-indexed. Art. 14: 24 hours.

World 2: Vuln Intelligence

Same SBOM, Two Scanners

677 PURL components from insecure-app → Grype (CPE + advisory DB) vs OSV (PURL-native)

Grype

141 unique vulnerabilities

122 packages with vulns
27 exclusive to Grype
114 shared with OSV

OSV

226 unique vulnerabilities

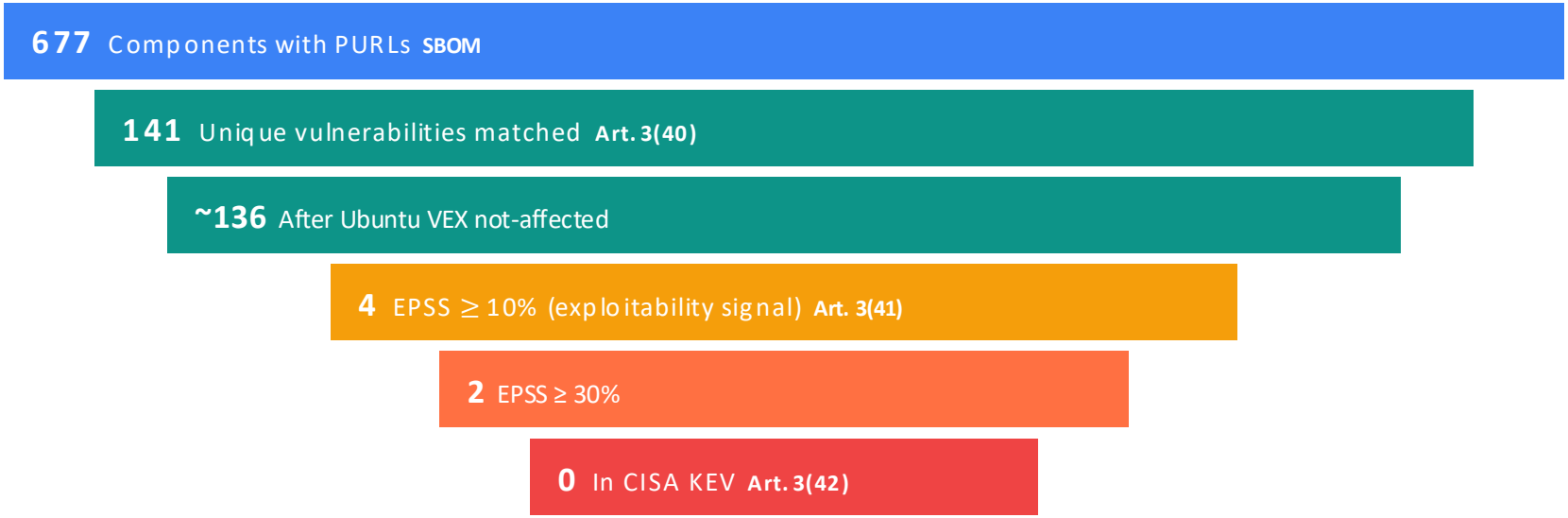
51 packages with vulns
112 exclusive to OSV
114 shared with Grype

253 total unique vulns — only 114 overlap. Different databases, different matching logic, different results. You have to become opinionated about which scanner to trust — and be ready to defend that choice under CRA.

Grype even reports 2 CVEs rejected in the CVE List (6 findings across ncurses + perl). OSV correctly filters them out. Your scanner's advisory DB may not.

The Enrichment Funnel

Grype scan of insecure-app: 677 packages → 141 unique vulns → CRA triage



Exploit PoCs: 99 of 141 CVEs have ≥1 public exploit PoC. Not in the ENISA §4.4.2 triage metrics (CVSS, EPSS, KEV, VEX) — already baked into EPSS as a feature.

141 vulns → ~136 after VEX → 2 with EPSS ≥ 30% → 0 in KEV → 0 that trigger Art. 14.

The Two That Survived Triage

ENISA Advisory §4.4.2: "treat CVSS ≥ 7.0 or EPSS ≥ 0.3 as high priority" — first EU-agency EPSS threshold

CVE-2023-0286 **cryptography 3.3.2** **EPSS 0.88** P99

X.400 address type confusion in X.509 GeneralName — OpenSSL under the hood. Grype finds it as GHSA, it as GHSA, maps to CVE. Both scanners agree. 0 public exploit PoCs — EPSS is based on other signals. signals.

CVE-2024-34069 **Werkzeug 3.0.1** **EPSS 0.39** P97

Debugger RCE when debug mode is enabled — the Flask app's own framework dependency. Fixed in 3.0.3. 0 public exploit PoCs — high EPSS without public PoC.

The GHSA-only blind spot: 5 findings have only a GHSA ID (no CVE). No CVE → no EPSS, no KEV check, no exploit DB lookup. You lose 3 lookup. You lose 3 of 4 ENISA §4.4.2 assessment metrics. The identifier gap isn't just a labelling issue — it's a triage blind spot. blind spot.

VEX Reality Check

Source	VEX?	Coverage
Red Hat	Yes — CSAF VEX	Full advisories
Ubuntu	Yes — OpenVEX	56/58 deb CVEs have status
Debian	No	—
PyPI / npm / Go	Nothing	81 vulns unfiltered

Ubuntu VEX Breakdown

needed: 17
released (patched): 12
deferred: 7
needs-triage: 7
DNE: 6
not-affected: 5
ignored: 2

Art. 14(8): User notification must be in "structured, machine-readable format." CSAF/VEX is the delivery mechanism. For app-layer ecosystems, you're generating VEX yourself.

Closing

Bridging Two Worlds

Each enrichment step from the demo maps to a CRA obligation

SBOM World		CRA World
Generate SBOM (677 PURLs) *ENISA SBOM Guide: 80% min transitive coverage*	→	**Annex I Part II §1**
Scan with Grype + OSV (141 vulns) *Comm. Guidance §211: EUVD + CVE List*	→	**Art. 13(7) third-party DB**
Apply Ubuntu VEX (~136 remain) *Comm. Guidance §213: must investigate applicability*	→	**Art. 3(41) exploitability**
EPSS triage (2 above 30%) *ENISA Advisory §4.4.2: EPSS ≥ 0.3 = high priority*	→	**Art. 13(5) due diligence**
KEV + threat intel (0 in KEV) *Comm. Guidance §195: reasonable certainty*	→	**Art. 3(42) → Art. 14 reporting**

Sources will contradict. You will have to be opinionated. Your CRA defence is in the process, not the number.

The CRA doesn't just want you to have an SBOM.

It wants you to operate a pipeline that continuously traverses both worlds — from component identity to exploitation signal — and surfaces actionable intelligence in time to matter.

This is not a finish line. It’s a treadmill.

New CVEs every day. New scanners, new VEX sources, new threat intel.
The pipeline you build in September is the one you’ll operate forever.

Thank you!

Andrey Lukashenkov
la@vulners.com
linkedin.com/in/alukashenkov

Scripts & materials: **github.com/alukashenkov/sbom-focus**

vulners.com